Laboratório: contratos Avro com Java e Spring Boot

Em 9 laboratórios curtos, você monta um produtor e um consumidor de sinais GPS que conversam pelo Kafka com contrato Avro, e depois quebra e conserta esse contrato de propósito para ver cada regra funcionando.

Visão geral

O projeto tem três partes:

- **produtor-sinais:** uma API REST que recebe uma posição e publica no tópico `sinais-posicao`. Faz o papel do ccpd-signals.
- **consumidor-alertas:** lê o tópico e registra um alerta quando a velocidade passa de 120 km/h. Faz o papel do ccpd-reactive.
- **Infraestrutura local:** Kafka, Schema Registry e uma interface web, todos via Docker Compose.

Lab	Pergunta que ele responde
0	Como subir Kafka e Schema Registry na minha máquina?
1	Como um arquivo `.avsc` vira uma classe Java?
2	O que de fato trafega no tópico e o que fica guardado no Registry?
3	Como produtor e consumidor em versões diferentes do contrato convivem?
4	O que acontece quando alguém tenta quebrar o contrato?
5	Por que o modo de compatibilidade define a ordem de deploy?
6	Como o consumidor sobrevive a uma mensagem que não consegue ler?
7	Como tirar o registro de esquemas das mãos dos serviços e passar para o pipeline?

Lab	Pergunta que ele responde
8	Como compartilhar contratos entre repositórios? (opcional)

Pré-requisitos: JDK 21, Maven 3.9 ou superior, Docker com Compose, `curl` e `jq`. Tempo estimado: 8 a 12 horas, em sessões de 1 a 2 horas.

Versões sugeridas: Spring Boot 3.5.x, Confluent Platform 7.7.1 (imagens Docker e serializador) e Avro 1.11.3. O roteiro não foi executado de ponta a ponta; se alguma versão não resolver, ajuste para a mais próxima compatível e anote a mudança.

Estrutura de pastas. Cada serviço tem a sua própria cópia do `.avsc` de propósito: isso deixa visível que nada se propaga sozinho (Lab 3). O Lab 8 mostra a alternativa com biblioteca compartilhada.

```
lab-avro/
├── docker-compose.yml
├── NOTAS.md
├── produtor-sinais/
│   ├── pom.xml
│   └── src/main/
│       ├── avro/SinalPosicao.avsc
│       ├── java/br/com/lab/produtor/...
│       └── resources/application.yml
└── consumidor-alertas/
    ├── pom.xml
    └── src/main/
        ├── avro/SinalPosicao.avsc
        ├── java/br/com/lab/consumidor/...
        └── resources/application.yml
```

Use o `NOTAS.md` para responder a pergunta de reflexão ao fim de cada lab. Esse arquivo, junto com o repositório, é o entregável do PDI.

Lab 0 – Ambiente local

Um único `docker compose up` sobe o Kafka (modo KRaft, sem Zookeeper), o Schema Registry e uma interface web para olhar tópicos e esquemas.

1. Crie a pasta `lab-avro/` e, dentro dela, o arquivo `docker-compose.yml`:

```
services:
  kafka:
    image: confluentinc/cp-kafka:7.7.1
    hostname: kafka
    container_name: kafka
    ports:
      - "9092:9092"
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:29093
      KAFKA_LISTENERS:
PLAINTEXT://kafka:29092,CONTROLLER://kafka:29093,PLAINTEXT_HOST://0.0.0.0:909
2
      KAFKA_ADVERTISED_LISTENERS:
PLAINTEXT://kafka:29092,PLAINTEXT_HOST://localhost:9092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      CLUSTER_ID: MkU3OEVBNTcwNTJENDM2Qk

  schema-registry:
    image: confluentinc/cp-schema-registry:7.7.1
    hostname: schema-registry
    container_name: schema-registry
    depends_on:
      - kafka
    ports:
      - "8081:8081"
    environment:
      SCHEMA_REGISTRY_HOST_NAME: schema-registry
      SCHEMA_REGISTRY_KAFKASTORE_BOOTSTRAP_SERVERS: kafka:29092
      SCHEMA_REGISTRY_LISTENERS: http://0.0.0.0:8081

  kafka-ui:
    image: ghcr.io/kafbat/kafka-ui:latest
    container_name: kafka-ui
    depends_on:
      - kafka
      - schema-registry
    ports:
```

```
      - "8085:8080"
    environment:
      KAFKA_CLUSTERS_0_NAME: lab
      KAFKA_CLUSTERS_0_BOOTSTRAPSERVERS: kafka:29092
      KAFKA_CLUSTERS_0_SCHEMAREGISTRY: http://schema-registry:8081
```

2. Suba tudo com `docker compose up -d` e espere cerca de 30 segundos.
3. Confirme que o Registry respondeu: `curl -s localhost:8081/subjects` deve devolver `[]` (nenhum esquema ainda).
4. Abra `http://localhost:8085` e veja o cluster `lab`, ainda sem tópicos.

Por que dois endereços para o Kafka? Os containers falam entre si por `kafka:29092`; as aplicações Spring Boot, rodando na sua máquina, usam `localhost:9092`. Isso aparece em todos os comandos a seguir.

Reflexão: em qual tópico interno o Schema Registry guarda os esquemas? (Dica: liste os tópicos na interface web.) O que aconteceria com os contratos se esse tópico fosse apagado?

Lab 1 – Projetos e primeiro contrato

O contrato é um arquivo `.avsc`; um plugin do Maven o transforma em uma classe Java no build, e essa classe compilada é a versão do contrato que cada serviço conhece.

1. No start.spring.io, gere dois projetos Maven com Java 21 e Spring Boot 3.5.x, grupo `br.com.lab`:
   a. `produtor-sinais`, com as dependências Spring Web e Spring for Apache Kafka.
   b. `consumidor-alertas`, só com Spring for Apache Kafka.
2. Em cada `pom.xml`, adicione as propriedades, o repositório da Confluent (o serializador não está no Maven Central), as dependências e o plugin do Avro:

```
<properties>
  <java.version>21</java.version>
  <confluent.version>7.7.1</confluent.version>
  <avro.version>1.11.3</avro.version>
</properties>

<repositories>
  <repository>
    <id>confluent</id>
    <url>https://packages.confluent.io/maven/</url>
  </repository>
</repositories>
```

```
<!-- dentro de <dependencies>, junto com o que o Initializr gerou -->
<dependency>
  <groupId>io.confluent</groupId>
  <artifactId>kafka-avro-serializer</artifactId>
  <version>${confluent.version}</version>
</dependency>
<dependency>
  <groupId>org.apache.avro</groupId>
  <artifactId>avro</artifactId>
  <version>${avro.version}</version>
</dependency>

<!-- dentro de <build><plugins> -->
<plugin>
  <groupId>org.apache.avro</groupId>
  <artifactId>avro-maven-plugin</artifactId>
  <version>${avro.version}</version>
  <executions>
    <execution>
      <phase>generate-sources</phase>
      <goals><goal>schema</goal></goals>
      <configuration>
        <sourceDirectory>${project.basedir}/src/main/avro</sourceDirectory>
        <outputDirectory>${project.build.directory}/generated-
sources/avro</outputDirectory>
        <stringType>String</stringType>
      </configuration>
    </execution>
  </executions>
</plugin>
```

3. Crie `src/main/avro/SinalPosicao.avsc` **nos dois projetos, com conteúdo idêntico**. Esta é a versão 1 do contrato:

```
{
  "type": "record",
  "name": "SinalPosicao",
  "namespace": "br.com.lab.contratos",
  "doc": "Posição GPS de um veículo ou agente na faixa de dutos",
  "fields": [
    {"name": "idObjeto",   "type": "string", "doc": "Placa do veículo ou
matrícula do agente"},
    {"name": "latitude",   "type": "double"},
    {"name": "longitude",  "type": "double"},
    {"name": "timestamp",  "type": {"type": "long", "logicalType":
"timestamp-millis"}},
    {"name": "velocidade", "type": ["null", "double"], "default": null,
"doc": "km/h; opcional"}
  ]
}
```

4. Rode `mvn compile` em cada projeto e abra `target/generated-sources/avro/br/com/lab/contratos/SinalPosicao.java`. Observe três coisas:
   a. O campo estático `SCHEMA$`: o esquema inteiro vai embutido na classe. É ele que o consumidor usa como esquema de leitura.
   b. O `newBuilder()`, que obriga a preencher os campos sem valor padrão.
   c. O tipo de `timestamp`: o tipo lógico vira `java.time.Instant`, e não um `long` solto.

Por que o `namespace` precisa ser igual nos dois projetos? O consumidor usa o nome completo do esquema (`br.com.lab.contratos.SinalPosicao`) para encontrar a classe Java correspondente. Nomes diferentes fazem a leitura cair para um registro genérico, sem a classe gerada.

Reflexão: se alguém alterar o `.avsc` e não recompilar o serviço, qual versão do contrato o serviço usa em produção? Onde, no SIPD, mora "a versão do contrato" de cada microsserviço?

Lab 2 – Produzir, consumir e inspecionar

Na primeira mensagem enviada, o serializador registra o esquema sozinho, recebe um ID e grava no tópico apenas esse ID seguido dos dados em binário, sem os nomes dos campos.

1. No produtor, configure `src/main/resources/application.yml`:

```
server:
  port: 8090
spring:
  application:
    name: produtor-sinais
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: io.confluent.kafka.serializers.KafkaAvroSerializer
    properties:
      schema.registry.url: http://localhost:8081
      auto.register.schemas: true
```

2. Crie o controller que publica no tópico. A chave da mensagem é o `idObjeto`, para que os sinais de um mesmo veículo caiam sempre na mesma partição e mantenham a ordem:

```
package br.com.lab.produtor;

import br.com.lab.contratos.SinalPosicao;
import java.time.Instant;
import org.springframework.http.ResponseEntity;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/sinais")
public class SinalController {

  private static final String TOPICO = "sinais-posicao";
  private final KafkaTemplate<String, SinalPosicao> kafka;

  public SinalController(KafkaTemplate<String, SinalPosicao> kafka) {
    this.kafka = kafka;
  }

  @PostMapping
  public ResponseEntity<Void> publicar(@RequestBody SinalRequest req) {
    SinalPosicao sinal = SinalPosicao.newBuilder()
        .setIdObjeto(req.idObjeto())
        .setLatitude(req.latitude())
        .setLongitude(req.longitude())
        .setTimestamp(Instant.now())
        .setVelocidade(req.velocidade())
```

```
        .build();
    // join() deixa o envio síncrono de propósito: erros de contrato aparecem
na resposta HTTP
    kafka.send(TOPICO, sinal.getIdObjeto(), sinal).join();
    return ResponseEntity.accepted().build();
  }

  public record SinalRequest(String idObjeto, double latitude, double
longitude, Double velocidade) {}
}
```

3. No consumidor, configure o `application.yml`. A propriedade `specific.avro.reader: true` faz o deserializador devolver a classe gerada, e não um registro genérico:

```
spring:
  application:
    name: consumidor-alertas
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: consumidor-alertas
      auto-offset-reset: earliest
      key-deserializer:
org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer:
io.confluent.kafka.serializers.KafkaAvroDeserializer
    properties:
      schema.registry.url: http://localhost:8081
      specific.avro.reader: true
```

4. Crie o listener:

```
package br.com.lab.consumidor;

import br.com.lab.contratos.SinalPosicao;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.stereotype.Component;

@Component
public class SinalListener {

  private static final Logger log =
LoggerFactory.getLogger(SinalListener.class);
```

```
  @KafkaListener(topics = "sinais-posicao")
  public void consumir(SinalPosicao sinal) {
    log.info("Recebido: {}", sinal);
    Double velocidade = sinal.getVelocidade();
    if (velocidade != null && velocidade > 120) {
      log.warn("ALERTA: {} a {} km/h", sinal.getIdObjeto(), velocidade);
    }
  }
}
```

5. Suba os dois com `mvn spring-boot:run` (um terminal para cada) e envie um sinal:

```
curl -s -X POST localhost:8090/sinais -H 'Content-Type: application/json' \
  -d
'{"idObjeto":"ABC1D23","latitude":-22.90,"longitude":-43.17,"velocidade":135}
'
```

O consumidor deve registrar a mensagem e o alerta de velocidade.

6. Inspecione o que ficou no Registry:

```
curl -s localhost:8081/subjects                                          #
["sinais-posicao-value"]
curl -s localhost:8081/subjects/sinais-posicao-value/versions            # [1]
curl -s localhost:8081/subjects/sinais-posicao-value/versions/1 | jq     # id,
versão e o esquema
curl -s localhost:8081/config                                            # modo
de compatibilidade global
```

7. Veja os bytes crus da mensagem no tópico:

```
docker exec kafka kafka-console-consumer --bootstrap-server kafka:29092 \
  --topic sinais-posicao --from-beginning --max-messages 1 | xxd | head -n 4
```

Os 5 primeiros bytes devem ser `00 00 00 00 01`: o byte "mágico" (0) e o ID do esquema (1) em 4 bytes. No resto, só aparece texto legível em `ABC1D23`; os nomes dos campos não viajam.

8. Agora veja a mesma mensagem decodificada pelo esquema:

```
docker exec -it schema-registry kafka-avro-console-consumer --bootstrap-
server kafka:29092 \
  --topic sinais-posicao --from-beginning --property
schema.registry.url=http://localhost:8081
```

Repare que `velocidade` aparece como `{"double": 135.0}`: é assim que o Avro representa em JSON um campo que pode ser `null` ou `double`. Saia com Ctrl+C.

Reflexão: compare o tamanho da mensagem com o equivalente em JSON. Se o Registry cair, um produtor que já está rodando continua enviando? E um que acabou de subir? (Dica: o serializador guarda os esquemas em cache.)

Lab 3 – Evolução compatível

Adicionar um campo com valor padrão é uma mudança segura: produtor e consumidor podem ficar em versões diferentes do contrato pelo tempo que for preciso.

1. Com os dois serviços rodando, altere **apenas o `.avsc` do produtor**, acrescentando um campo ao final da lista:

```
{"name": "precisao", "type": ["null", "double"], "default": null, "doc":
"Precisão do GPS, em metros"}
```

2. No produtor, inclua `Double precisao` no `SinalRequest` e `.setPrecisao(req.precisao())` no builder.
3. Antes de subir, pergunte ao Registry se a mudança é compatível com a última versão registrada:

```
jq -n --rawfile s produtor-sinais/src/main/avro/SinalPosicao.avsc '{schema:
$s}' > /tmp/v2.json
curl -s -X POST -H 'Content-Type: application/vnd.schemaregistry.v1+json' \
  --data @/tmp/v2.json localhost:8081/compatibility/subjects/sinais-posicao-
value/versions/latest
# {"is_compatible":true}
```

4. Reinicie só o produtor e envie um sinal com `"precisao": 4.5`. Confira:
   a. `curl -s localhost:8081/subjects/sinais-posicao-value/versions` agora devolve `[1,2]`.
   b. Os bytes da nova mensagem começam com `00 00 00 00 02`.
   c. O consumidor, ainda na versão 1, continua funcionando e simplesmente não mostra `precisao`.
5. Agora atualize o consumidor: copie o mesmo `.avsc` da versão 2 para ele. Pare o consumidor e volte o grupo para o início do tópico, para reler tudo:

```
docker exec kafka kafka-consumer-groups --bootstrap-server kafka:29092 \
  --group consumidor-alertas --reset-offsets --to-earliest --topic sinais-
posicao --execute
```

6. Suba o consumidor de novo. As mensagens antigas aparecem com `precisao` nulo (o valor padrão) e as novas com 4.5.

As quatro combinações que você acabou de testar:

Esquema de escrita (produtor)	Esquema de leitura (consumidor)	Resultado
v1	v1	Leitura normal
v2	v1	`precisao` é ignorado
v1	v2	`precisao` recebe o valor padrão (`null`)
v2	v2	`precisao` preenchido

Reflexão: quem precisou de deploy para o consumidor passar a usar a precisão? Que mudança real do SIPD seria parecida (por exemplo, uma rastreadora que começa a enviar um campo novo)?

Lab 4 – Mudança incompatível barrada

Renomear um campo quebra o contrato, e o Registry recusa a nova versão no próprio produtor, antes de qualquer mensagem chegar ao tópico.

1. No `.avsc` do produtor, renomeie `latitude` para `lat` e ajuste o builder (`.setLat(...)`).
2. Teste a compatibilidade como no Lab 3. A resposta agora é `{"is_compatible":false}`. Acrescente `?verbose=true` ao final da URL para ver o motivo, se a sua versão do Registry suportar.
3. Reinicie o produtor e envie um sinal. O `curl` recebe erro 500, e o log do produtor mostra uma `RestClientException` com código 409: o esquema que se tentou registrar é incompatível com uma versão anterior.
4. Confira o que **não** aconteceu: as versões no Registry continuam `[1,2]`, nenhuma mensagem nova entrou no tópico e o consumidor nem percebeu.
5. Desfaça a mudança no produtor antes de seguir.

Exercício de previsão. Para cada mudança abaixo, anote no `NOTAS.md` se acha que o modo BACKWARD aceita, e só depois confira no endpoint de compatibilidade:

- [ ] Mudar o tipo de `latitude` de `double` para `string`.
- [ ] Mudar o tipo de um campo `int` para `long` (crie um campo `int` com valor padrão para testar).

- [ ] Remover o campo `velocidade`, que tem valor padrão.
- [ ] Adicionar um campo `bateria` do tipo `int`, **sem** valor padrão.

Reflexão: em qual serviço o erro apareceu? Por que é melhor o produtor falhar ao enviar do que o consumidor descobrir o problema ao ler?